

กลุ่ม 17 — Life times

สมาชิกผู้จัดทำ

1. นายภารร นานช้า (670710631) — Concept + Code
2. นายรัชชานนท์ เจริญรมย์ (670710632) — Code + Demo
3. นาย ราบีล แดใหญ่ (670710633) — Rust vs Other Language + PPL
4. นาย วีรศิลป์ สีนาวงศ์ (670710634) — Exercises + Common Mistakes

Learning Objectives

หลังจากศึกษา Topic นี้แล้ว ผู้เรียนสามารถ:

1

อธิบายแนวคิดสำคัญได้

2

เขียนโปรแกรม Rust ที่เกี่ยวข้องได้

3

วิเคราะห์พฤติกรรม/กฎของภาษาได้

4

เปรียบเทียบ Rust กับภาษาอื่นได้

Introduction

การจัดการหน่วยความจำในภาษาโปรแกรมโดยทั่วไปแบ่งเป็นแบบจัดการเอง (C/C++) ที่รวดเร็วแต่เสี่ยงช่องโหว่ และแบบใช้ Garbage Collector (Java, Go) ที่ปลอดภัยแต่แลกมาด้วยความล่าช้า (Overhead) ภาษา Rust จึงทลายข้อจำกัดนี้ด้วยการจัดการหน่วยความจำตั้งแต่ขั้นตอนการคอมไพล์ผ่าน 2 แนวคิดหลัก ได้แก่:

Lifetime (อายุขัยของข้อมูล)

คอมไพเลอร์จะตรวจสอบขอบเขตเวลาใช้งานของทุกการอ้างอิง (Reference) เพื่อป้องกันปัญหา Dangling Pointers (พอยน์เตอร์ชี้ค้าง) โดยมีกฎเหล็กคือตัวอ้างอิงต้องไม่มีอายุขัยยาวนานกว่าตัวข้อมูลจริง

Ownership (ระบบความเป็นเจ้าของ)

ประยุกต์ใช้ตรรกะ Affine Type System ที่กำหนดให้ข้อมูลแต่ละชิ้นมีเจ้าของได้เพียงตัวเดียวเท่านั้น เมื่อตัวแปรเจ้าของสิ้นสุดการทำงาน (Out of scope) ระบบจะแทรกคำสั่งคืนค่าหน่วยความจำให้โดยอัตโนมัติ

ผลลัพธ์ที่ได้คือ Zero-cost Abstraction ที่การันตีความปลอดภัยของหน่วยความจำได้ 100% โดยไม่ต้องพึ่งพา Garbage Collector ทำให้โปรแกรมมีความเร็วและประสิทธิภาพเทียบเท่าภาษา C หรือ C++

4.1 Shared XOR Mutable

Borrow Checker คือกลไกของคอมไพเลอร์ที่บริหารจัดการการยืมข้อมูล เพื่อตรวจสอบว่าการเข้าถึงหน่วยความจำในทุก ๆ ตำแหน่งสอดคล้องกับกฎของการยืมหรือไม่ เป้าหมายหลักในขั้นต้นคือเพื่อป้องกันปัญหา Data Races (การแย่งกันแก้ไขข้อมูลพร้อมกัน)

Shared XOR Mutable ในช่วงเวลาหนึ่ง ข้อมูลสามารถมีตัวอ้างอิงแบบอ่านอย่างเดียว (Immutable) ได้หลายตัว หรือ มีตัวอ้างอิงแบบแก้ไขได้ (Mutable) เพียง 1 ตัว เท่านั้น ห้ามมีทั้งสองแบบพร้อมกันโดยเด็ดขาด

```
fn main() {  
    let mut data = String::from("Rust");  
  
    let r1 = &data; // ยืมแบบอ่าน  
    let r2 = &data; // ยืมแบบอ่านตัวที่สอง (ทำได้)  
  
    let r3 = &mut data; // ยืมแบบแก้ไข (Error: ทำไม่ได้เพราะ r1, r2 ยังใช้งานอยู่)  
  
    println!("Read: {} and {}", r1, r2);  
}
```

Explanation

เมื่อตัวแปร r1 และ r2 กำลังถือสิทธิ์การอ่านแบบ Shared อยู่ Borrow Checker จะไม่อนุญาตให้ r3 ขอสิทธิ์แบบ Mutable เพื่อเข้ามาแก้ไขข้อมูลเด็ดขาดจนกว่า r1 และ r2 จะทำงานเสร็จสิ้น เพื่อรับประกันว่าจะไม่มีใครแอบเปลี่ยนข้อมูลในขณะที่คนอื่นกำลังอ่านอยู่

4.2 Non-Lexical Lifetimes (NLL)

เพื่อให้การจัดการหน่วยความจำยืดหยุ่นขึ้นและป้องกัน

ปัญหา Use-after-free

Borrow Checker จะทำงานอยู่บน Region Variables โดยวิเคราะห์ กราฟเส้นทางการทำงานของโปรแกรม (Control Flow Graph - CFG) แทนที่จะดูแค่ขอบเขตของโค้ด

ด้วย NLL อายุขัยของการอ้างอิงจะสิ้นสุดลงทันทีเมื่อ สิ้นสุดการเรียกใช้งานจริงในบรรทัดสุดท้าย ไม่ต้องรอให้จบโครงสร้างบล็อกปิดกั้น {} เพื่อลดการรายงานข้อผิดพลาดที่เข้มงวดเกินไป

```
fn main() {  
    let mut x = 10;  
  
    let y = &mut x; // เริ่มยืมแบบแก้ไข  
    *y += 5;        // y ถูกใช้งานครั้งสุดท้ายที่บรรทัดนี้  
  
    // ด้วย NLL คอมไพเลอร์รู้ทันทีว่า y จบหน้าที่แล้ว  
    // จึงอนุญาตให้ z เข้ามายืม x ต่อได้โดยไม่ต้องรอให้จบปีกกา  
    let z = &x;  
    println!("x is now: {}", z);  
}
```

Explanation

กลไก CFG ของคอมไพเลอร์ติดตามเส้นทางการใช้งานของตัวแปร y และพบว่ามันไม่ได้ถูกใช้งานอีกเลยหลังจากบรรทัด `y += 5;`

Borrow Checker จึงปลดล็อกสิทธิ์การยืมทันที ทำให้ z สามารถอ้างอิงตัวแปร x ต่อได้โดยไม่ผิดกฎ Shared XOR Mutable

4.3 Explicit Lifetime Annotation

มักจะถูกเขียนแทนด้วยสัญลักษณ์ Apostrophe ตามด้วยชื่อตัวแปรขนาดเล็ก เช่น 'a (อ่านว่า ทิก-เอ)
แม้ว่าในหลายฟังก์ชัน คอมไพเลอร์จะสามารถอนุมานอายุขัยของตัวแปรภายในฟังก์ชัน (Intra-procedural analysis) ได้อย่างแม่นยำโดยไม่ต้องพึ่งพาไวยากรณ์เหล่านี้ แต่เมื่อโปรแกรมมีความซับซ้อนมากขึ้น โดยเฉพาะเมื่อมีการรับค่าอ้างอิงเข้ามาหลายตัวและมีการคืนค่าอ้างอิงกลับออกไป การระบุ Lifetime จึงกลายเป็นสิ่งจำเป็นอย่างหลีกเลี่ยงไม่ได้ ใช้เพื่อเชื่อมโยงว่าค่าอ้างอิง (Reference) ขาออกนั้น มีความเกี่ยวข้องกับค่าอ้างอิงขาเข้าตัวใด
การระบุ Lifetime ไม่ได้ช่วยยืดอายุขัยของข้อมูลจริง แต่เป็นเพียงการอธิบายตรรกะให้ Borrow Checker มั่นใจว่าจะไม่มีการเข้าถึงข้อมูล (Dangling pointer) หลังจากที่เราของข้อมูลนั้นถูกทำลายไปแล้ว

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() {  
        x  
    } else {  
        y  
    }  
}
```

Explanation

เครื่องหมาย 'a เป็นค่า Reference ที่ฟังก์ชันนี้ส่งคืนกลับไป จะมีอายุการใช้งานเท่ากับตัวแปรขาเข้า (x หรือ y) ตัวที่มีอายุสั้นที่สุด เพื่อให้คอมไพเลอร์ตรวจสอบฟังก์ชันผู้เรียก (Caller) ได้อย่างปลอดภัยโดยไม่ต้องเข้ามาวิเคราะห์โค้ดข้างในฟังก์ชันนี้ซ้ำ

Important Syntax / Rules

Syntax / Rule	Meaning	Example
&i32	ค่าอ้างอิงแบบอ่านได้อย่างเดียว (Immutable reference) ที่ถูกซ่อนรูป Lifetime ไว้ โดยคอมไพเลอร์จัดการอายุขัยให้เองโดยอัตโนมัติ ตามกฎ Lifetime Elision	fn print_data(x: &i32)
&'a i32	ค่าอ้างอิงแบบอ่านได้อย่างเดียว ที่ผู้เขียนโปรแกรมระบุ Lifetime Parameter ชื่อ 'a อย่างชัดเจน เพื่อใช้จับคู่ความสัมพันธ์ของอายุขัยกับพารามิเตอร์หรือผลลัพธ์อื่น	fn get_data<'a>(x: &'a i32) -> &'a i32
&'a mut i32	ค่าอ้างอิงแบบแก้ไขได้ (Mutable reference) ซึ่งถูกระบุ Lifetime เป็น 'a โดยจะถูกควบคุมอย่างเข้มงวดภายใต้กฎ Shared XOR Mutable ไม่ให้มีการอ้างอิงอื่นเข้ามาแทรกแซง	fn mutate_data<'a>(x: &'a mut i32)
&'static str	ข้อมูลคงที่ (Static) ที่ถูกจัดสรรลงในพื้นที่หน่วยความจำของไบนารีโดยตรง ข้อมูลประเภทนี้จะมีชีวิตอยู่ตลอดช่วงอายุการประมวลผลทั้งหมดของโปรแกรม	let msg: &'static str = "PPL";
struct Foo<'a>	โครงสร้างข้อมูล (Struct) ที่บรรจุค่าอ้างอิงไว้ภายในตัวมันเอง คอมไพเลอร์ต้องการการยืนยันว่าอายุขัยของ Struct จะไม่มีทางอยู่ยาวนานเกินกว่าอายุขัย 'a ของค่าที่มันเก็บรักษาไว้	struct Config<'a> { path: &'a str }
T: 'a	ไวยากรณ์ข้อจำกัด (Lifetime bound) บ่งบอกว่า ประเภทข้อมูลทั่วไป (Generic type T) ไດ ๆ จะต้องมียุขัยอย่างน้อยที่สุดครอบคลุมช่วงเวลา 'a จึงจะนำมาใช้งานได้	struct Ref<'a, T: 'a> { r: &'a T }

Important Rules

1. กฎของพารามิเตอร์ขาเข้าอิสระ (Independent Input Lifetimes)

พารามิเตอร์ขาเข้า (Input parameter) ทุกตัวที่เป็นค่าอ้างอิง จะได้รับ Lifetime parameter ที่ เป็นเอกเทศจากคอมไพเลอร์โดยอัตโนมัติ

2. กฎการถ่ายทอดจากพารามิเตอร์เดียว (Single Input Propagation)

หากมีพารามิเตอร์ขาเข้าที่เป็นค่าอ้างอิงเพียง ตัวเดียวเท่านั้น Lifetime ของพารามิเตอร์ ตัวนั้น จะถูกดึงไปกำหนดให้กับผลลัพธ์ขาออก (Output lifetimes) ทุกตัวทันที

3. กฎบริบทของโครงสร้างข้อมูลและเมธอด (Self's Lifetime Dominance)

ในบริบทของการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented) หากฟังก์ชันนั้นเป็นเมธอด ที่มีพารามิเตอร์รับค่าอ้างอิงหลายตัว แต่หนึ่งในนั้นคือ `&self` หรือ `&mut self` คอมไพเลอร์จะ อนุมานให้ Lifetime ของตัวแปร `self` ถูก กำหนดลงในผลลัพธ์ขาออกทั้งหมด

Example 1 — การเรียกใช้ตัวแปรที่หมดอายุแล้ว

```
fn main(){
    let x = 5;
    let a;
    {
        let y = 6;
        a = &y;
        print!("{}",a);
    }
    //println!("{}",x,a); **ถ้าเรียกข้างนอกขอบเขตอายุไขของ y จะหมดก่อน**
}
```

Purpose

ให้ดูว่า lifetime ในขอบเขตมีระยะเวลาเท่าใด

Expected Output

ได้ค่า 6

Explanation

เริ่มมาเราจะสร้างและกำหนดค่าของ x เป็น 5 จากนั้นสร้างตัวแปร a ขึ้นมาหลังจากนั้นเราจะกำหนดขอบเขตขึ้นมาแล้วสร้างและกำหนดค่าของ y ข้างในนั้นพร้อมทั้งนำค่า a ไป Reference ถึงค่า y ก็คือ 6 จากนั้นจะให้เรียก print a ออกมาจะได้ค่าที่เก็บไว้คือ 6(Reference จาก y)

Example 2 — lifetime ของ Function

```
fn higher<'a>(x: &'a i32, y: &'a i32) -> &'a i32 {
    if x > y{
        x
    }else{
        y
    }
}

fn main() {
    let x = 30;
    let result;
    {
        let y = 10;
        result = higher(&x,&y); // ใช้ได้ใน scope นี้
        println!("The higher value : {}", result);
    }
    //println!("The higher value : {}", result);
}
```

Purpose

ระยะเวลาและชื่อของ life time ของ Function

Expected Output

The higher value : 30

Explanation

เราจะสร้าง function higher ขึ้นมาก่อนเพื่อเช็คค่าไหนมากกว่ากันตัว function ก็จะได้รับ parameter มา 2 ตัวคือ x,y จากนั้นก็เทียบค่าว่าค่าใดมากกว่าและส่งค่าที่มากกว่าออกไป

ใน function main เราจะกำหนดค่า x = 30 และสร้างตัวแปร result ไว้ จากนั้นเราจะกำหนดขอบเขตขึ้นมาและสร้างตัวแปร y ขึ้นมาให้มีค่า 10 จากนั้นเราจะทำการกำหนดให้ result เก็บค่าที่เรียกใช้ function higher มาหลังจากนั้นให้ print ค่า result จะได้ค่าที่มากกว่าเป็นคำตอบ

Example 3 — lifetime ของ struct

```
struct Card<'a> {
    name: &'a str,
}

impl<'a> Card<'a> {
    fn show(&self) -> &str {
        self.name // return reference ที่มี lifetime เดียวกับ &self
    }
}

fn main() {
    let i;
    {
        let s = "FixHe4Rt".to_string();
        let username = &s;
        i = Card { name: username };
        println!("Name : {}",i.show());
    }
    //println!("Name : {}",i.show());
}
```

Purpose

lifetime ของตัวที่ Reference ถึง field ใน struct

Expected Output

Name : FixHe4Rt

Example 3 — lifetime ของ struct — Explanation

เริ่มจากการที่เราจะสร้าง struct ชื่อ Card ขึ้นมาพร้อมตั้งชื่อ lifetime ไว้ให้ด้วยและในนั้นจะมี field ชื่อ name จากนั้นเราจะสร้าง impl block เพื่อเพิ่ม method ให้กับ Card พร้อมตั้งชื่อ lifetime เดียวกันกับ struct และข้างในจะมี method ชื่อ show และมี self คือตัวแทนของ instance ที่เรียก method นั้นอยู่ และให้ output ออกมาเป็น self.name หรือก็คือ name ที่เก็บไว้ใน struct และ ใน Function main เราจะกำหนดค่า s เป็น "FixHe4Rt" และ username คือตัวที่ Reference ถึง s และ สร้าง i เป็น Card ที่เก็บ name เป็น username และเรียกใช้ print i.show() ซึ่ง Rust จะส่ง i เข้าไปเป็น self โดยอัตโนมัติ ฟังก์ชันเลยเข้าถึง self.name ได้ จึงได้ค่าออกมาเป็นค่าที่เก็บไว้ใน name ของ Card ถ้าในส่วนที่คอมเมนต์อยู่นำไปใช้งานแทนจะทำให้เกิด error เพราะ lifetime ของ s หหมดลงทำให้ถ้าไปเรียกข้างนอก block i.name(self.name) ที่ชี้ไปตำแหน่งที่ s เคยอยู่จะเกิด dangling reference

Mistake 1 — Dangling Pointer

พยายามดึงข้อมูลที่ถูก Drop ไปแล้ว

✗ Incorrect

```
fn dangle() -> &String
{
    let s = String::from("hi");
    &s
} // s หหมด scope ตรงนี้ ค่า "hi" จะถูก drop ทั้งเพื่อคืนพื้นที่หน่วยความจำ
fn main()
{
    let a = dangle(); //อิงถึง s แล้วไม่เจอค่าเพราะถูก drop ไปแล้ว ทำให้ Compile ไม่ผ่าน

    println!("{}", a);
}
```

✓ Correct

```
fn no_dangle() -> String
{
    let s = String::from("hi");
    s
} //ส่งออก Ownership "hi" ให้ตัวแปรที่เรียก function
fn main()
{
    let a = no_dangle(); //a เป็นเจ้าของค่า "hi" แทน s แล้ว

    println!("{}", a);
}
```

ทำไมถึงผิด? (Why?)

เนื่องจากใน Code ที่ผิดเป็นการเข้าถึงค่าผ่านการ Reference ซึ่งใน Rust หากตัว Owner หลุด Scope ไปแล้วค่าที่ตัว Owner own อยู่ จะถูก Drop ทั้งเพื่อคืนพื้นที่หน่วยความจำ ทำให้เมื่อจบ dangle() ตัวแปร s ซึ่งเป็น Owner ของค่า "hi" นั้นหลุด scope ไปแล้ว โปรแกรมจึง drop ค่า "hi" ทั้ง เมื่อ a พยายามอิงถึงค่าภายใน s จึงมองไม่เห็นอะไรทำให้ compile ไม่ผ่าน แต่ใน code ที่ถูกต้อง no_dangle() ส่งออก Ownership แทนการ reference เฉยๆ ทำให้ a เป็น Owner ของ "hi" แทน s จึงสามารถใช้งานได้ปกติ

Mistake 2 — Lifetime Specifier

ไม่ได้กำหนด Lifetime specifier ให้ตัวที่จำเป็น

✗ Incorrect

```
struct Highlight
{
    text: &str,    // จะ Compile ไม่ผ่านตั้งแต่ตรงนี้
}

fn main()
{
    let content = String::from("Rust is fun");
    let h = Highlight { text: &content };
    println!("{}", h.text);
}
```

✓ Correct

```
struct Highlight<'a>
{
    text: &'a str, //มีการเพิ่ม 'a เข้ามา
}

fn main()
{
    let content = String::from("Rust is fun");
    let h = Highlight { text: &content };
    println!("{}", h.text);
}
```

ทำไมถึงผิด? (Why?)

เนื่องจาก Struct ที่เก็บค่าโดยการ Reference Rust ไม่สามารถทราบได้ว่าตัวแปรที่ถูกอิงถึงนั้นจะมีอายุอยู่พอสำหรับตลอดการเรียกใช้งานของ Struct หรือไม่ เพื่อป้องกัน Dangling pointers Rust จึงบังคับให้ประกาศ Lifetime specifier ('a ในที่นี้แทนความยาวของ Lifetime หนึ่งที่ชื่อ a) เป็นตัวช่วยในตอน Compile เพื่อยืนยันว่า Lifetime ของตัวที่ถูกอิงถึงจะมีอายุยืนพอตลอดระยะเวลาที่สามารถเรียกใช้งาน Struct ได้ โดยถ้าตรวจสอบแล้วว่า Owner อาจตายก่อน Code จะไม่ผ่านตั้งแต่ตอน Compile

Exercise 1 — Let me out!

Problem

ให้เขียนโปรแกรมที่ประกาศตัวแปรข้อความว่า "I'm in the block" ไว้หนึ่งตัวภายใน block { } จากนั้นนำค่าของตัวแปรนั้นไปพิมพ์ภายนอก block ให้ได้



Hint

ต้องทำให้ข้อความภายใน Block มีอายุต่อหลังจากออกจาก Block ให้ได้

```
fn main()
{
    let outer;
    {
        let inner = String::from("I'm in the block"); // inner เป็นเจ้าของ "I'm in the block" อยู่
        outer = inner; // ย้าย ownership ของ "I'm in the block" ให้ outer ซึ่งถูกประกาศอยู่ main
    }
    println!("{}", outer);
}
```

Explanation

inner เป็นตัวแปรที่เป็นเจ้าของ "I'm in the block" ซึ่งถูกสร้างภายใน block หากพยายาม print ตัว inner โดยตรงหรือให้ outer เก็บค่าผ่านการอิง inner (&inner) แล้ว print outer จะไม่สามารถทำได้ เพราะหลังจากออก Block "I'm in the block" จะถูก Drop เพราะ inner ซึ่งเป็น Owner ถูกโปรแกรมมองว่าหลุด scope ไปแล้วทำให้ใน inner จะไม่มีค่าเก็บอยู่ (Dangling pointer) แก้ปัญหาได้โดยการย้ายความเป็นเจ้าของ (Ownership) ของ "I'm in the block" จาก inner สู่ outer แทน ทำให้โปรแกรม Run ได้ตามปกติ

Exercise 2 — Which one's longer?

Problem

ให้เขียนฟังก์ชันชื่อ `longer` ที่รับข้อความ 2 ค่าเข้ามาผ่านการอ้างอิง (reference) แล้วคืนค่าข้อความที่ยาวกว่าออกไป (ถ้ายาวเท่ากันคืนค่าไหนก็ได้) จากนั้นเขียน `main` ที่เรียกใช้ฟังก์ชันนี้และแสดงผลให้ถูกต้อง



Hint

Rust ทราบหรือไม่ว่าต้อง Return ตัวไหน ? แล้วทั้ง 2 ตัวจำเป็นต้องมีอายุเท่ากันหรือไม่ ?

```
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str //กำหนด 'a เพื่อให้ rust เช็คอายุของทั้ง 2 string
{
    if x.len() > y.len()
    {
        x
    }
    else
    {
        y
    }
}

fn main()
{
    let a = String::from("Pine");
    let b = String::from("Apple");
    let result = longer(&a, &b); //โยน reference เข้าไป
    println!("ยาวกว่าคือ: {}", result);
}
```

Explanation

เนื่องจาก `longer()` รับตัวแปรเข้าไป 2 ตัวและต้องผ่านกระบวนการ `if else` เพื่อตัดสินว่า `a` หรือ `b` จะถูก `return` กลับมา ทำให้ `rust` ไม่สามารถเดาได้ว่าตัวไหนกันแน่ที่จะถูก `return` มา ซึ่งอาจมีปัญหาดังกล่าวหากตัวที่จะถูก `return` เกิดมีอายุสั้นเกินไป (ในกรณีนี้เช่น `let b = String::from("Apple");` ซึ่งเป็นตัวที่จะ `return` ถูกประกาศภายใน `block` แล้ว `print` ที่นอก `block` ทำให้ค่า `"Apple"` ถูก `drop` ไปก่อน) ทำให้ต้องมีการประกาศ `Lifetime specifier ('a)` เพื่อให้ `rust` ตรวจสอบและมั่นใจว่าทุกตัวแปรที่ส่งเข้าไปจะมีอายุอยู่ยาวนานพอในขณะที่ `longer()` ยังสามารถถูกเรียกใช้ได้อยู่

9.1 Syntax

`fn main()` เป็นการประกาศฟังก์ชันหลักของโปรแกรม

- `{ ... }` ใช้กำหนดขอบเขตของฟังก์ชันหรือบล็อกคำสั่ง
- `;` ใช้จบคำสั่ง

รูปแบบการประกาศตัวแปร:

- `let` ใช้ประกาศตัวแปร
- `mut` ระบุว่าตัวแปรสามารถเปลี่ยนแปลงค่าได้
- `: i32` ใช้ระบุชนิดข้อมูลเป็นจำนวนเต็ม 32 บิต

`println!("{}", number)` เป็น macro สำหรับแสดงผลของ Rust โดย `{}` เป็นช่องสำหรับใส่ค่าของตัวแปร

9.2 Semantics

ในตัวอย่างนี้ `fn main()` เป็นจุดเริ่มต้นของโปรแกรม ส่วน `let` ใช้สร้าง Binding ระหว่างชื่อตัวแปรกับค่าเริ่มต้น 1 และ `mut` อนุญาตให้ค่าในตัวแปรเปลี่ยนแปลงได้ คำสั่ง `number += 2` มีความหมายเทียบเท่ากับ `number = number + 2` จึงทำให้ค่าของ `number` เปลี่ยนจาก 1 เป็น 3 ขณะที่ `println!` ทำหน้าที่แสดงค่าดังกล่าวออกทางหน้าจอ ดังนั้น ตัวอย่างนี้เลยแสดงลำดับพฤติกรรมของโปรแกรม ตั้งแต่เริ่มทำงาน สร้างตัวแปร เปลี่ยนค่าตัวแปร และแสดงผลเป็น 3

```
fn main() {  
    let mut number: i32 = 1;  
    number += 2;  
    println!("{}", number);  
}
```

9.3 Type System

Rust เป็นภาษาที่มีคุณสมบัติด้าน Type System ดังนี้:

Static typing

ตรวจสอบชนิดข้อมูลตอน Compile

Strong typing

ไม่ยอมให้ชนิดข้อมูลที่ไม่เข้ากัน
ทำงานร่วมกันโดยไม่ตรวจสอบ

Type inference

บางกรณี Rust สามารถอนุมานชนิด
ข้อมูลให้เองได้

เช่น การเขียน `let mut number = 1` ทำให้ Rust อนุมานชนิดข้อมูลของ `number` เป็นจำนวนเต็ม ซึ่งสามารถเขียนแบบระบุชนิดข้อมูลได้เป็น `let mut number: i32 = 1`
ถ้าเขียน `number += 2.5` จะเกิด Error เพราะ `number` เป็น `i32` แต่ `2.5` เป็นเลขทศนิยม Rust จึงตรวจพบความผิดพลาดตั้งแต่ตอน Compile

9.4 Memory / Resource Management

ในตัวอย่างนี้ ตัวแปร `number` มีชนิดเป็น `i32` ซึ่งเป็นข้อมูลขนาดเล็กและถูกเก็บไว้บน Stack โดยมีขอบเขตการใช้งานอยู่ภายในฟังก์ชัน `main()`

เมื่อจบการทำงานของฟังก์ชัน หน่วยความจำของตัวแปรจะถูกคืนให้อัตโนมัติ ไม่จำเป็นต้องเรียกคำสั่งสำหรับลบหน่วยความจำเอง

ในภาษา C หากใช้ `malloc()` จะต้องใช้ `free()` เองเพื่อคืนหน่วยความจำ แต่โดยทั่วไป Rust จะจัดการคืนหน่วยความจำให้เองตาม Ownership และ Scope หากต้องการปล่อยข้อมูลก่อนเวลา สามารถใช้ `drop()` ได้

Rust ใช้แนวคิด Ownership และ Scope ในการจัดการหน่วยความจำ โดยตรวจสอบระหว่าง Compile เพื่อป้องกันการใช้ข้อมูลหลังหมดอายุหรือการเข้าถึงหน่วยความจำที่ไม่ถูกต้อง สำหรับตัวอย่างนี้ไม่มีการจองหน่วยความจำบน Heap หรือการใช้ Resource ภายนอกโดยตรง

ตัวอย่างการจองหน่วยความจำบน Heap:

`Box::new(10)` ใช้สร้างค่า 10 บน Heap ส่วน `drop(number)` ใช้ปล่อยค่าออกก่อนจบฟังก์ชัน อย่างไรก็ตาม หากไม่เขียน `drop(number)` Rust จะปล่อยหน่วยความจำให้อัตโนมัติเมื่อ `number` หมด Scope

```
fn main() {  
    let number = Box::new(10);  
    println!("{}", number);  
  
    drop(number);  
}
```

9.5 Abstraction / Other PPL Concepts

Binding

การผูกชื่อเข้ากับค่า ชนิดข้อมูล หรือฟังก์ชัน
เช่น `let number = 1`

Scope

ขอบเขตที่ชื่อหรือตัวแปรสามารถใช้งานได้ เช่น
ภายในฟังก์ชันหรือบล็อก `{ ... }`

Abstraction

การซ่อนรายละเอียดการทำงานและรวมคำสั่งไว้ในหน่วยที่เรียกใช้งานง่าย เช่น ฟังก์ชัน, `struct` และ `enum`

Ownership

ค่าทุกค่ามีเจ้าของ และเมื่อเจ้าของหมด Scope Rust จะคืนทรัพยากรให้อัตโนมัติ

Borrowing

การให้ส่วนอื่นยืมข้อมูลผ่านการอ้างอิงด้วย & โดยไม่โอน Ownership

Lifetime

ระยะเวลาที่ข้อมูลหรือ Reference สามารถใช้งานได้ Rust ใช้ตรวจสอบไม่ให้ Reference อยู่ได้นานกว่าข้อมูลที่อ้างอิงถึง

Type System

Rust ตรวจสอบชนิดข้อมูลและความเข้ากันได้ของการดำเนินการตั้งแต่ตอน Compile

Generics

การเขียนฟังก์ชันหรือโครงสร้างข้อมูลให้รองรับหลายชนิดข้อมูลโดยไม่ต้องเขียนซ้ำ

Traits

การกำหนดพฤติกรรมร่วมที่ชนิดข้อมูลสามารถนำไปใช้ได้ คล้ายแนวคิด Interface

Pattern Matching

การตรวจสอบและแยกรูปแบบข้อมูลด้วย `match` และรูปแบบอื่น ๆ

Paradigm

Rust รองรับหลายกระบวนทัศน์ เช่น Imperative, Functional และ Object-oriented บางส่วน โดยผู้เขียนสามารถสั่งงานเป็นลำดับ ใช้ฟังก์ชันเป็นค่า และสร้างชนิดข้อมูลที่มีพฤติกรรมได้

9.6 Why Rust?

Rust ใช้แนวคิดเหล่านี้เพื่อเพิ่มความปลอดภัย ความน่าเชื่อถือ และประสิทธิภาพของโปรแกรม ดังนี้:

Safety

Static typing และ Strong typing ช่วยตรวจพบการใช้ชนิดข้อมูลผิดตั้งแต่ตอน Compile ส่วน Ownership, Borrowing และ Lifetime ช่วยป้องกันการใช้น้อย ความจำหลั้หมดอายุและการเข้าถึงข้อมูลที่ไม่ถูกต้อง

Reliability

Rust บังคับให้ผู้เขียนจัดการ Scope และความสัมพันธ์ระหว่างข้อมูลอย่างชัดเจน ทำให้ลดข้อผิดพลาด เช่น Memory Leak, Double Free และ Data Race

Performance

Rust จัดการหน่วยความจำโดยไม่ต้องใช้ Garbage Collector ทำให้ควบคุมทรัพยากรได้ดีและมีประสิทธิภาพใกล้เคียงภาษาระดับต่ำ เช่น C หรือ C++

Zero-cost abstractions

Abstraction เช่น Generics และ Traits สามารถใช้ได้โดยไม่ต้องแลกกับประสิทธิภาพขณะรันโปรแกรมอย่างมีนัยสำคัญ

Compile-time checking

Rust ตรวจสอบปัญหาสำคัญหลายอย่างก่อนรันจริง ทำให้โปรแกรมมีโอกาสทำงานผิดพลาดน้อยลง

10.1 Comparison Table

เลือกเปรียบเทียบ Rust กับ Go, Zig และ Swift เนื่องจากแต่ละภาษามีลักษณะคล้าย Rust ในบางด้าน เช่น Static typing, Performance หรือการพัฒนาโปรแกรมระบบ

Aspect	Rust	Go	Zig	Swift
Syntax	ใช้ let, mut, fn และ Macro เช่น println!	ใช้ func, := และคำสั่ง go สำหรับ Concurrency	ใช้ Syntax เรียบง่าย เน้น Explicit control และมี defer	ใช้ let, var, Function และ Property พร้อม Syntax ที่อ่านง่าย
Semantics / Behavior	ตรวจสอบ Ownership และ Borrowing ตอน Compile	ใช้ Goroutine และ Channel พร้อม Garbage Collector	ควบคุมการทำงานใกล้ Hardware และจัดการ Error แบบ Explicit	ใช้ Value semantics และตรวจสอบ Memory ด้วย ARC
Type System	Static และ Strong typing พร้อม Type inference	Static typing พร้อม Type inference และ Interface	Static typing พร้อม Compile-time checking และ Optional	Static และ Strong typing พร้อม Type inference และ Protocol
Memory Management	จัดการอัตโนมัติด้วย Ownership โดยทั่วไปไม่ใช้ Garbage Collector	ใช้ Garbage Collector จัดการหน่วยความจำ	ให้ผู้พัฒนาควบคุมหน่วยความจำอย่างชัดเจน โดยไม่มี Garbage Collector	ใช้ Automatic Reference Counting (ARC)
Safety	ป้องกันหลายปัญหาตั้งแต่ Compile เช่น Use-after-free และ Data Race	มี Memory Safety จาก Garbage Collector แต่ตรวจบางปัญหาตอน Runtime	มีเครื่องมือช่วยตรวจสอบ แต่ผู้พัฒนายังต้องระวัง Pointer	ลดปัญหา Memory ด้วย ARC และตรวจสอบชนิดข้อมูลตอน Compile
Performance	สูงและเหมาะกับงานระบบ	สูงและเหมาะกับงาน Concurrent Server	สูงและควบคุมทรัพยากรได้ใกล้เคียง C	สูง เหมาะกับงาน Application และระบบของ Apple

10.2 – 10.5 Same Program in Four Languages

Rust

```
fn main() {  
    let mut number: i32 = 1;  
    number += 2;  
    println!("{}", number);  
}
```

Go

```
package main  
  
import "fmt"  
  
func main() {  
    number := 1  
    number += 2  
    fmt.Println(number)  
}
```

Zig

```
const std = @import("std");  
  
pub fn main() !void {  
    var number: i32 = 1;  
    number += 2;  
    try std.io.getStdOut().writer().print("{d}\n", .{number});  
}
```

Swift

```
var number: Int = 1  
number += 2  
print(number)
```

10.6 Analysis

ความคล้ายและความต่าง

Rust, Go, Zig และ Swift มีบางลักษณะที่คล้ายกัน เช่น เป็นภาษา Static typing และเหมาะกับงานที่ต้องการ Performance แต่ไม่ได้เหมือนกันทุกด้าน โดย Go เน้นการพัฒนา Concurrent Application ที่ง่ายขึ้นด้วย Goroutine และ Garbage Collector ส่วน Zig เน้นการควบคุมทรัพยากรและความเรียบง่ายใกล้เคียงภาษาระดับระบบ ขณะที่ Swift ใช้ ARC เพื่อช่วยจัดการหน่วยความจำและมีระบบชนิดข้อมูลที่ปลอดภัย

แนวทางด้าน Memory Safety

แม้ทั้งสามภาษาจะมีลักษณะคล้าย Rust แต่แนวทางด้าน Memory Safety แตกต่างกัน Rust ใช้ Ownership, Borrowing และ Lifetime ตรวจสอบการจัดการหน่วยความจำตั้งแต่ตอน Compile โดยไม่ต้องใช้ Garbage Collector ขณะที่ Go ใช้ Garbage Collector, Zig ให้ผู้พัฒนาควบคุมหน่วยความจำเอง และ Swift ใช้ ARC

สรุป

ดังนั้น Rust จึงโดดเด่นด้วยการผสาน Performance ระดับภาษา System Programming เข้ากับ Memory Safety ที่ตรวจสอบได้ตั้งแต่ Compile ทำให้เหมาะกับงานที่ต้องการทั้งความเร็วและความน่าเชื่อถือ

References

Rust & Lifetimes

1. The Rust Programming Language — <https://doc.rust-lang.org/nomicon/lifetimes.html>
2. Rustify - <https://rustify.rs/articles/rust-lifetimes-deep-dive-2026>
3. Rust, from first principles | Alejandro Soto Franco (<https://www.sotofranco.dev/articles/posts/rust-formal-semantics>)
4. Mastering Lifetimes in Rust: Memory Safety and Borrow Checking (<https://dev.to/leapcell/mastering-lifetimes-in-rust-memory-safety-and-borrow-checking-1ge6>)

Go · Zig · Swift

5. The Go Authors. The Go Programming Language. <https://go.dev/>
6. Zig Software Foundation. Zig Language Reference. <https://ziglang.org/documentation/master/>
7. Apple Inc. The Swift Programming Language. <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>

AI Usage Declaration

สามารถใช้ AI เป็นเครื่องมือช่วยเรียนรู้และพัฒนาได้ แต่สมาชิกทุกคนต้องเข้าใจและสามารถอธิบายผลงานของกลุ่มได้

AI Tool	Purpose	How the Result Was Verified
Claude	ทำความเข้าใจเกี่ยวกับหัวข้อ และช่วยแนะแนวในการทำเนื้อหาบางส่วนเพื่อให้ข้อมูลถูกต้องตามหลักที่สุด	ตรวจสอบกับแหล่งข้อมูลอื่น + ลองนำโค้ดมารัน
Chatgpt	ใช้ร่วมกับ Claude เพื่อตรวจสอบกันเอง	ตรวจสอบกับแหล่งข้อมูลอื่น + ลองนำโค้ดมารัน
Gemini Deep Research	ใช้สืบค้นข้อมูลเชิงลึกภาษา Rust เพื่อร่างเนื้อหาและทำสไลด์นำเสนอ Lifetime	ตรวจสอบความถูกต้องของแนวคิดเทียบกับเอกสารอ้างอิงและทดสอบรันโค้ดจริง
Gemini Notebook	ใช้จัดระเบียบข้อมูล วิเคราะห์ความถี่การอ้างอิง และรวมประเด็นสำคัญจากเอกสารต้นฉบับ	ตรวจสอบความสอดคล้องของผลลัพธ์ AI สรุป กับเนื้อหาจริงในเอกสารต้นฉบับ ว่าไม่มีการสร้างข้อมูลเท็จ

Declaration

- ✓ Code ทุกส่วนที่นำเสนอได้รับการ Compile และทดสอบแล้ว
- ✓ สมาชิกทุกคนสามารถอธิบาย Code ที่นำเสนอได้
- ✓ ตรวจสอบข้อมูลจากแหล่งอ้างอิงที่น่าเชื่อถือแล้ว
- ✓ ระบุการใช้ AI อย่างโปร่งใส

รายละเอียดการใช้ AI

ใช้ในการทำความเข้าใจ lifetime และช่วยแนะแนวทางในการทำเกี่ยวกับบาง Topic โดยมีการตรวจสอบข้อมูลผ่านเว็บไซต์ชั้นนำเพื่อให้แน่ใจว่า AI ให้ข้อมูลอย่างถูกต้อง และไม่บิดเบือน
ใช้ในขั้นตอนค้นหาเรื่อง กฎ Shared XOR Mutable, Non-Lexical Lifetimes (NLL), Explicit Lifetime Annotation และ Borrow Checker และกฎต่างๆ

GitHub Contribution

Member	Issues	Commits	Pull Requests	Code Reviews	Contribution
นาย ภารร นานข้า	0	12	2	0	1648++ 661--
นาย รัชชานนท์ เจริญรมย์	0	14	5	0	245++ 241--
นาย ราโมล แดใหญ่	0	7	1	0	243++ 72--
นาย วีรศิลป์ สีนาวงศ์	0	26	8	0	1699++ 1041--

Teamwork Reflection

How did your team collaborate?

พูดคุยผ่านโปรแกรม Discord & Line และทำไฟล์งานร่วมกันผ่านเว็บ Github

Problems encountered

ใช้งานบาง Features ของ Github ยังไม่คล่อง ส่งผลการทำงานให้ล่าช้าในช่วงแรก และ ไม่เข้าใจเนื้อหาบางส่วนเกี่ยวกับหัวข้อที่ได้รับ

How did you solve them?

หาข้อมูลผ่าน internet เช่น Google, Youtube, AI และหารี้อร่วมกับเพื่อนในส่วนที่พยายามทำความเข้าใจร่วมกัน